University of Illinois Urbana-Champaign

ECE 408: Applied Parallel Programming

ECE 408 CNN Milestone 3 Report

Performance Engineering and Optimization Analysis

Student: Ashmit Dutta
NetID: ashmitd2

Final Submission File:
`project/src/layer/custom/m3-forward.cu`

April 28, 2026

Abstract

This report documents the final ECE 408 Milestone 3 CNN submission and the optimization process that led to a competition-safe, single-stream implementation. The final path combines a shape-specialized implicit-GEMM Conv1 kernel with a WMMA Tensor Core Conv2 kernel that performs fused implicit unrolling plus tiled matrix multiplication. At batch 10000, the final source achieves 12.54609 ms total convolution operator time with 0.8714 accuracy, staying well below both the full-credit performance threshold and the competition warmup thresholds.

Contents

1. Submission Artifacts

Google Drive profiling folder: INSERT_GOOGLE_DRIVE_M3_FOLDER_LINK_HERE.

Final code submission: `project/src/layer/custom/m3-forward.cu`

Required report filename for Gradescope: `ECE408_SP26_ashmitd2_m3_report.pdf`.

1.1 Submission Checklist Before Final Upload

- GitHub: final performance file committed at `project/src/layer/custom/m3-forward.cu`.
- GitHub: each individual optimization committed under `project/m3/req_#` or `project/m3/op_#` with a non-stacked `m3-forward.cu`.
- Google Drive: copied template `m3` folder, granted Viewer access to Google Apps @ Illinois, and uploaded profiling binaries for each implemented optimization.
- Google Drive: included one subfolder per implemented optimization: `req_0`, `req_1`, `op_0`, `op_1`, `op_2`, `op_3`, `op_5`, and `op_6`.
- Gradescope: uploaded PDF named `ECE408_SP26_ashmitd2_m3_report.pdf`.
- Gradescope: assigned pages to optimization numbers during submission.

1.2 Google Drive Subfolder Links

- `req_0`: INSERT_REQ_0_DRIVE_LINK_HERE.
- `req_1`: INSERT_REQ_1_DRIVE_LINK_HERE.
- `op_0`: INSERT_OP_0_DRIVE_LINK_HERE.
- `op_1`: INSERT_OP_1_DRIVE_LINK_HERE.
- `op_2`: INSERT_OP_2_DRIVE_LINK_HERE.
- `op_3`: INSERT_OP_3_DRIVE_LINK_HERE.
- `op_5`: INSERT_OP_5_DRIVE_LINK_HERE.
- `op_6`: INSERT_OP_6_DRIVE_LINK_HERE.

1.3 Implemented Optimization Coverage

The milestone report asks each optimization section to include the optimization name/number, theory, implementation details, code evidence, performance-vs-expectation analysis, synergy with other optimizations, and references. The sections below are organized around those required elements for `req_0`, `req_1`, `op_0`, `op_1`, `op_2`, `op_3`, `op_5`, and `op_6`.

2. Executive Summary

The final submitted source is a single-stream, competition-safe implementation. Conv1 uses a shape-specialized implicit-GEMM kernel with constant-memory weights and four-column thread coarsening. Conv2 uses a tiled implicit-GEMM Tensor Core kernel through CUDA WMMA: the Conv2 mask is treated as matrix A, the input is gathered as an implicit im2col matrix B, and the

Figure 1: Final submission data path. Conv1 uses a specialized scalar implicit-GEMM kernel, while Conv2 uses a tiled WMMA implicit-GEMM kernel to satisfy the competition matmul requirement.

output is matrix C. The final source does not launch CUDA streams, and all GPU kernel launches occur inside `conv_forward_gpu()`.

All reported final runs match the expected baseline accuracy. In particular, the batch-10000 final run and the `--competition` mode run both report 0.8714 accuracy, preserving final-submission and competition eligibility.

The final source is well below the M3 full-credit target of 60 ms combined convolution op time. It also satisfies the competition warmup guardrail that each convolution layer must stay below 40 ms.

3. Baselines and Final Selection

The final selection is the specialized WMMA kernel because it gives the best validated timing among the submitted implicit-GEMM, tiled-matmul variants while preserving the expected accuracy.

4. Optimization Methodology

The optimization process was measurement-driven. I optimized Conv1 and Conv2 separately, used batch-100 smoke tests to reject incorrect or obviously slower variants, and then used batch-10000 runs for promising candidates. The Conv2 work focused on tiled implicit-GEMM with WMMA Tensor Cores, so every final-candidate measurement stayed aligned with the submitted kernel family.

Table 1: Final competition-safe performance summary.

Run	Batch	Conv1 op time	Conv2 op time	Total conv op time	Accuracy
Final normal run	100	0.085260 ms	0.137248 ms	0.222508 ms	0.86
Final normal run	10000	2.76322 ms	9.78287 ms	12.54609 ms	0.8714
Final `--competition` mode	10000	2.87406 ms	9.71168 ms	12.58574 ms	0.8714

Table 2: Baselines and final implementation selection.

Implementation	Batch	Conv1 op time	Conv2 op time	Total conv op time	Notes
M1 GPU baseline	10000	11.63 ms	52.61 ms	64.24 ms	Earlier project baseline
M2 fused baseline	10000	51.57 ms	31.03 ms	82.59 ms	Baseline for most individual optimizations
WMMA baseline	100	0.084228 ms	0.156062 ms	0.240290 ms	Early WMMA reference implementation
WMMA candidate A	10000	2.83016 ms	10.1206 ms	12.95076 ms	Intermediate WMMA block/tile configuration
Final specialized WMMA	10000	2.76322 ms	9.78287 ms	12.54609 ms	Final competition-safe source

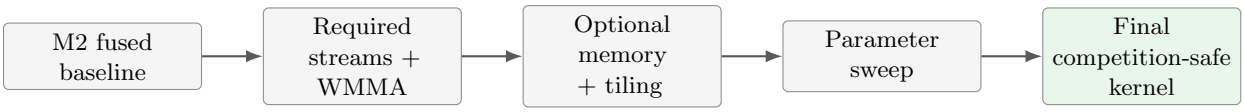

Figure 2: Optimization workflow. Individual required and optional optimizations were measured separately, then the final source selected the optimizations that improved the submitted WMMA implicit-GEMM path.

4.1 Analytical Model Used During Tuning

I modeled both convolution layers in implicit-GEMM form:

$$C_{M\times N} = A_{M\times K} \cdot B_{K\times N}, \quad \text{FLOPs} = 2MNK$$

where:

- M = number of output channels or filters,
- $K = C_{in} \cdot K_h \cdot K_w$, the reduction dimension,
- $N = B \cdot H_{out} \cdot W_{out}$, the number of output pixels across the batch.

For this milestone, the fixed dimensions are:

- Conv1: $M = 4$, $K = 1 \cdot 7 \cdot 7 = 49$, $N = 10000 \cdot 80 \cdot 80 = 64{,}000{,}000$
- Conv2: $M = 16$, $K = 4 \cdot 7 \cdot 7 = 196$, $N = 10000 \cdot 34 \cdot 34 = 11{,}560{,}000$

This gives:

- Conv1 compute: $2 \cdot 4 \cdot 49 \cdot 64{,}000{,}000 = 25.088$ GFLOPs
- Conv2 compute: $2 \cdot 16 \cdot 196 \cdot 11{,}560{,}000 = 72.504$ GFLOPs

I also used a simple memory-bandwidth lower bound with A40 DRAM peak bandwidth of 696 GB/s:

$$t_{bw} \geq \frac{\text{bytes moved}}{\text{peak BW}}$$

- Conv1 minimum, input plus output only: $(296 + 1024)$ MB / 696 GB/s ≈ 1.9 ms

- Conv2 minimum, input plus output only: $(256 + 740)$ MB / 696 GB/s ≈ 1.4 ms

Comparing this floor to measured batch-10000 kernel times:

- Conv1: $2.76322/1.9 \approx 1.45\times$ above the bandwidth floor, so it is near memory-optimal.
- Conv2: $9.78287/1.4 \approx 6.99\times$ above the bandwidth floor, so it remains the dominant bottleneck.

Arithmetic Intensity View

$$\text{AI} = \frac{\text{FLOPs}}{\text{Bytes moved}} \quad \text{FLOPs/byte.}$$

Using the same lower-bound byte counts, input plus output only:

- Conv1: $\text{AI}_{min} \approx \frac{25.088\times10^9}{1.320\times10^9} \approx 19.0$ FLOPs/byte
- Conv2: $\text{AI}_{min} \approx \frac{72.504\times10^9}{0.996\times10^9} \approx 72.8$ FLOPs/byte

These are optimistic upper-bound AI values because they ignore additional traffic from implicit gathers, shared-memory staging, and synchronization overhead. In the real kernel, effective AI is lower, especially for Conv2 where each WMMA step requires non-trivial index decode and staged B-tile construction.

At batch 10000, achieved arithmetic throughput is:

- Conv1: $\frac{25.088\,\text{GFLOPs}}{2.76322\,\text{ms}} \approx 9.08\,\text{TFLOP/s}$
- Conv2: $\frac{72.504\,\text{GFLOPs}}{9.78287\,\text{ms}} \approx 7.41\,\text{TFLOP/s}$

For Conv2 WMMA specifically:

$$K_{tiles} = \left\lceil \frac{196}{16} \right\rceil = 13, \quad N_{tiles} = \left\lceil \frac{11{,}560{,}000}{16} \right\rceil = 722{,}500$$

So Conv2 executes about $13 \cdot 722{,}500 = 9{,}392{,}500$ WMMA tile updates, plus implicit-im2col gather and output scatter work.

5. Conv1 Optimization

Conv1 has fixed dimensions in this project: `Map_out=4`, `Channel=1`, `K=7`, and output size `80x80`. The mask has only $4 \cdot 1 \cdot 7 \cdot 7 = 196$ weights, so I place it in module-scope constant memory. The kernel performs implicit-GEMM without creating an explicit im2col buffer.

Thread mapping is:

$$\text{tid} \mapsto (b, h_{out}, w_{out}^{base}), \quad w_{out}^{base} = 4 \cdot \left\lfloor \frac{w_{out}}{4} \right\rfloor$$

Each thread computes four neighboring output columns, with coarsening factor 4, for all 4 output channels. Accumulators are kept in registers:

$$y[m, h_{out}, w_{out} + c] = \sum_{p=0}^{6} \sum_{q=0}^{6} w[m, p, q]\; x[h_{out} + p, w_{out} + c + q], \quad c \in \{0, 1, 2, 3\}.$$

```cpp
static constexpr int kConv1MapOut    = 4;
static constexpr int kConv1KernelDim = 7;
static constexpr int kConv1Ckk       = kConv1KernelDim * kConv1KernelDim;
static constexpr int kCoarsen        = 4;
__constant__ float c_mask[16384];

__global__ void Conv1ImplicitGemmKernel(const float* __restrict__ input,
                                        float* __restrict__ output,
                                        int batch, int map_out, int channel,
                                        int height, int width, int k_dim) {
    const int height_out      = height - k_dim + 1;
    const int width_out       = width  - k_dim + 1;
    const int width_coarsened = (width_out + kCoarsen - 1) / kCoarsen;
    const int tid = blockIdx.x * blockDim.x + threadIdx.x;

    const int total_positions = batch * height_out * width_coarsened;
    if (tid >= total_positions) return;

    const int b          = tid / (height_out * width_coarsened);
    const int rem        = tid % (height_out * width_coarsened);
    const int h_out      = rem / width_coarsened;
    const int w_out_base = (rem % width_coarsened) * kCoarsen;

    float acc[kConv1MapOut][kCoarsen] = {0.0f};

    #pragma unroll
    for (int k = 0; k < kConv1Ckk; ++k) {
        const int p = k / kConv1KernelDim;
        const int q = k - p * kConv1KernelDim;

        float x_val[kCoarsen];
        #pragma unroll
        for (int cw = 0; cw < kCoarsen; ++cw) {
            const int w_out = w_out_base + cw;
            x_val[cw] = (w_out < width_out)
                ? __ldg(&input[(size_t)b * height * width
                               + (h_out + p) * width + w_out + q])
                : 0.0f;
        }

        #pragma unroll
        for (int m = 0; m < kConv1MapOut; ++m) {
            const float w = c_mask[m * kConv1Ckk + k];
            #pragma unroll
            for (int cw = 0; cw < kCoarsen; ++cw) {
                acc[m][cw] += w * x_val[cw];
            }
        }
    }
    #pragma unroll
    for (int m = 0; m < kConv1MapOut; ++m) {
        const size_t out_base = static_cast<size_t>(b) * map_out * height_out * width_out
                              + static_cast<size_t>(m) * height_out * width_out
                              + static_cast<size_t>(h_out) * width_out;
        #pragma unroll
        for (int cw = 0; cw < kCoarsen; ++cw) {
            const int w_out = w_out_base + cw;
            if (w_out < width_out) {
                output[out_base + w_out] = acc[m][cw];
            }
        }
    }
}
```

Listing 1: Final Conv1 implicit-GEMM kernel excerpt: constant-memory weights, four-column coarsening, and register accumulation.

Table 3: Conv1 optimization candidates and selection.

Conv1 candidate	Batch	Conv1 op time	Accuracy	Decision
M2 fused baseline	10000	51.57 ms	0.8714	Baseline
Final coarsened implicit-GEMM Conv1	10000	2.76322 ms	0.8714	Kept in final source

The final Conv1 kernel uses constant-memory broadcast for the weights, no explicit unrolled matrix, and four-column coarsening. This combination gave a large speedup while keeping the Conv1 path simple enough to pair cleanly with the final WMMA Conv2 kernel.

6. Conv2 Competition-Safe WMMA Design

Conv2 has fixed dimensions after pooling: `Map_out=16`, `Channel=4`, `K=7`, input `40x40`, output `34x34`, and `CKK=196`. The final Conv2 kernel is `Conv2WmmaImplicitGemmKernel`.

- A: mask matrix $[16 \times 196]$, stored as FP16 for Tensor Core input.
- B: implicit im2col matrix $[196 \times (\text{Batch} \cdot 34 \cdot 34)]$, gathered into shared memory on the fly.
- C: output matrix $[16 \times (\text{Batch} \cdot 34 \cdot 34)]$, scattered back to NCHW output.

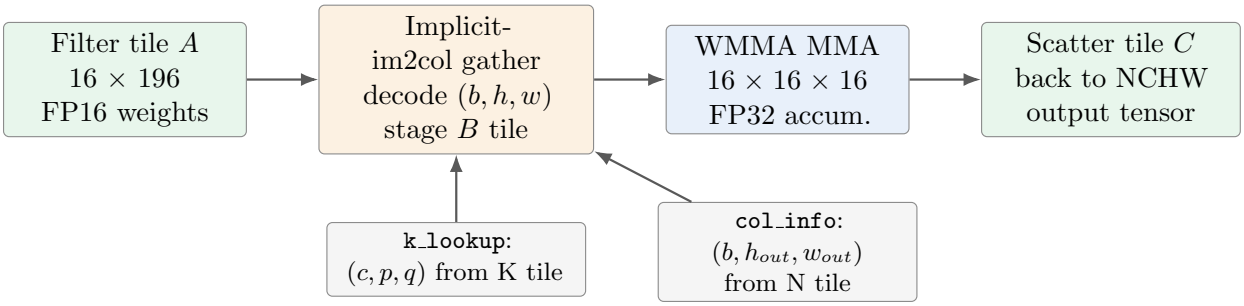

Figure 3: Conv2 fused implicit-GEMM mapping. The kernel avoids materializing the full im2col matrix by decoding each output-column coordinate and staging only the WMMA tile needed for the current warp.

Table 4: Final Conv2 WMMA tuning parameters.

Parameter	Final value	Reason
`kWmmaM`	16	Covers all Conv2 output maps in one WMMA M tile
`kWmmaN`	16	Native WMMA N tile
`kWmmaK`	16	Native WMMA K tile; K32-style variant was slower
`kWarpsPerBlock`	4	Best final WMMA setting among validated competition-safe variants
`kNTilesPerWarp`	1	Avoids extra register/shared-memory pressure from larger per-warp N tiling
Fixed Conv2 shape constants	enabled	Removes repeated runtime indexing work for fixed dimensions

```cpp
static constexpr int kWmmaM          = 16;
static constexpr int kWmmaN          = 16;
static constexpr int kWmmaK          = 16;
static constexpr int kWarpSize       = 32;
static constexpr int kWarpsPerBlock  = 4;
static constexpr int kNTilesPerWarp  = 1;
static constexpr int kConv2BlockSize = kWarpsPerBlock * kWarpSize;

__global__ void Conv2WmmaImplicitGemmKernel(const half*  __restrict__ mask_half,
                                            const float* __restrict__ input,
                                            float*       __restrict__ output,
                                            int map_out, int channel,
                                            int height, int width, int k_dim,
                                            int height_out, int width_out,
                                            int batch) {
    const int ckk         = channel * k_dim * k_dim;
    const int hw_out      = height_out * width_out;
    const int hw          = height * width;
    const int num_k_tiles = (ckk + kWmmaK - 1) / kWmmaK;
    const size_t n_total  = static_cast<size_t>(batch) * hw_out;

    const int warp_id = threadIdx.x / kWarpSize;
    const int lane_id = threadIdx.x % kWarpSize;
    const size_t block_tile_start = static_cast<size_t>(blockIdx.x)
                                  * kWarpsPerBlock * kNTilesPerWarp;

    __shared__ half smem_a[kWmmaM][kWmmaK];
    __shared__ half smem_b[kWarpsPerBlock][kNTilesPerWarp][kWmmaK][kWmmaN];
    __shared__ int  k_lookup[kWmmaK][3];
    __shared__ int  col_info[kWarpsPerBlock][kNTilesPerWarp][3][kWmmaN];

    const size_t n_tile = block_tile_start + warp_id;
    const size_t col_start = n_tile * kWmmaN;
    if (lane_id < kWmmaN) {
        const size_t col = col_start + lane_id;
        if (col < n_total) {
            const int b_idx = static_cast<int>(col / hw_out);
            const int s = static_cast<int>(col - static_cast<size_t>(b_idx) * hw_out);
            col_info[warp_id][0][0][lane_id] = b_idx;
            col_info[warp_id][0][1][lane_id] = s / width_out;
            col_info[warp_id][0][2][lane_id] = s % width_out;
        } else {
            col_info[warp_id][0][0][lane_id] = -1;
        }
    }
    __syncthreads();

    wmma::fragment<wmma::accumulator, kWmmaM, kWmmaN, kWmmaK, float> acc_frag;
    wmma::fragment<wmma::matrix_a, kWmmaM, kWmmaN, kWmmaK,
                   half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, kWmmaM, kWmmaN, kWmmaK,
                   half, wmma::row_major> b_frag;

    wmma::fill_fragment(acc_frag, 0.0f);

    for (int kt = 0; kt < num_k_tiles; ++kt) {
        const int k_start = kt * kWmmaK;

        if (threadIdx.x < kWmmaK) {
            const int k_idx = k_start + threadIdx.x;
            if (k_idx < ckk) {
                const int c   = k_idx / (k_dim * k_dim);
                const int rem = k_idx - c * k_dim * k_dim;
                k_lookup[threadIdx.x][0] = c;
                k_lookup[threadIdx.x][1] = rem / k_dim;
                k_lookup[threadIdx.x][2] = rem % k_dim;
            }
        }

        for (int i = threadIdx.x; i < kWmmaM * kWmmaK; i += kConv2BlockSize) {
            const int r = i / kWmmaK;
            const int c = i % kWmmaK;
            const int k_idx = k_start + c;
            smem_a[r][c] = (k_idx < ckk)
                ? mask_half[r * ckk + k_idx]
                : __float2half(0.0f);
        }
        __syncthreads();

        for (int i = lane_id; i < kWmmaK * kWmmaN; i += kWarpSize) {
            const int r = i / kWmmaN;
            const int n = i - r * kWmmaN;
            const int k_idx = k_start + r;
            half val = __float2half(0.0f);
            if (k_idx < ckk && col_info[warp_id][0][0][n] >= 0) {
                const int b_idx = col_info[warp_id][0][0][n];
                const int h_out = col_info[warp_id][0][1][n];
                const int w_out = col_info[warp_id][0][2][n];
```

```
                const int cc = k_lookup[r][0];
                const int p  = k_lookup[r][1];
                const int q  = k_lookup[r][2];
                val = __float2half(__ldg(&input[static_cast<size_t>(b_idx) * channel * hw
                                                + cc * hw
                                                + (h_out + p) * width
                                                + (w_out + q)]));
            }
            smem_b[warp_id][0][r][n] = val;
        }

        __syncwarp();
        wmma::load_matrix_sync(a_frag, &smem_a[0][0], kWmmaK);
        wmma::load_matrix_sync(b_frag, &smem_b[warp_id][0][0][0], kWmmaN);
        wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);
        __syncthreads();
    }

    __shared__ float smem_c[kWarpsPerBlock][kWmmaM][kWmmaN];
    wmma::store_matrix_sync(&smem_c[warp_id][0][0], acc_frag,
                            kWmmaN, wmma::mem_row_major);
    __syncwarp();

    for (int i = lane_id; i < kWmmaM * kWmmaN; i += kWarpSize) {
        const int m = i / kWmmaN;
        const int n = i - m * kWmmaN;
        if (col_info[warp_id][0][0][n] >= 0) {
            const int b_idx = col_info[warp_id][0][0][n];
            const int h_out = col_info[warp_id][0][1][n];
            const int w_out = col_info[warp_id][0][2][n];
            output[static_cast<size_t>(b_idx) * (map_out * hw_out)
                   + m * hw_out + h_out * width_out + w_out] = smem_c[warp_id][m][n];
        }
    }
}
```

Listing 2: Final Conv2 WMMA kernel excerpt: tile constants, shared-memory staging, implicit-im2col metadata, and WMMA fragments.

This excerpt is a compact reduction of the submitted kernel: it decodes output-column metadata, stages the filter tile and implicit-im2col tile, performs WMMA, and scatters the accumulator tile back to NCHW output.

7. Conv2 Parameter Sweep

7.1 WMMA and PTX MMA Sweep

The best valid path was the shape-specialized 4-warp WMMA kernel in the submitted source. I also explored PTX MMA variants to compare fragment mapping and shared-memory stride behavior, but the WMMA path remained the most robust accuracy-and-performance tradeoff for the final submission.

8. Profiling and Resource Evidence

The final Nsight Compute report for the WMMA Conv2 path is `m3_wmmaspec_conv2_ncu.ncu-rep`. It profiles the same WMMA kernel family used by the final source.

Nsight Compute shows that the final WMMA path materially improved SM utilization over the earlier Tensor Core version. Tensor-pipe activity remains low because this small Conv2 shape spends a large fraction of time on implicit im2col address generation, shared-memory staging, synchronization, and output scatter rather than pure MMA instructions.

`cuobjdump --dump-resource-usage` for the submitted final build reports no stack or local-memory spills. The Conv2 WMMA SASS resource line reports `REG:69 STACK:0 SHARED:7616 LOCAL:0`; Nsight Compute reports `56` launch registers/thread for the collected kernel instance. Conv1 reports no shared memory and no local-memory spills.

9. Required Optimizations

9.1 req_0: Streams

Folder: `project/m3/req_0`

Artifacts: `m3.out`, `req_0_ncu.ncu-rep`, `req_0_nsys.nsys-rep`, `req_0_nsys_stdout.log`

Google Drive subfolder: INSERT_REQ_0_DRIVE_LINK_HERE.

Theory: CUDA streams can overlap independent data transfers and kernel execution. The required stream optimization builds on the unfused input-unrolling path because that path naturally has separate unroll, tiled-matmul, and permute kernels.

Implementation: `req_0/m3-forward.cu` splits the batch into up to four chunks. For each chunk, it launches unrolling, `matrixMultiplyShared`, and permutation in a separate CUDA stream. The prolog also demonstrates asynchronous host-to-device copies.

```
#define NUM_STREAMS 4

const int stream_count = Batch < NUM_STREAMS ? Batch : NUM_STREAMS;
const int chunk_size = (Batch + stream_count - 1) / stream_count;

cudaStream_t streams[NUM_STREAMS];
float *unrolled[NUM_STREAMS] = {nullptr};
float *matmul_output[NUM_STREAMS] = {nullptr};

for (int s = 0; s < stream_count; ++s) {
    const int batch_start = s * chunk_size;
    const int current_batch = min(chunk_size, Batch - batch_start);
    if (current_batch <= 0) continue;

    cudaStreamCreate(&streams[s]);
    cudaMalloc(&unrolled[s], Height_unrolled * Width_unrolled * sizeof(float));
    cudaMalloc(&matmul_output[s], Map_out * Width_unrolled * sizeof(float));

    const float* input_chunk = device_input
        + static_cast<size_t>(batch_start) * Channel * Height * Width;
    float* output_chunk = device_output
        + static_cast<size_t>(batch_start) * Map_out * out_image_size;

    matrix_unrolling_kernel<<<unroll_grid, block_size, 0, streams[s]>>>(
        input_chunk, unrolled[s], current_batch, Channel, Height, Width, K);

    matrixMultiplyShared<<<matmul_grid, matmul_block, 0, streams[s]>>>(
        device_mask, unrolled[s], matmul_output[s],
        Map_out, Height_unrolled, Height_unrolled, Width_unrolled,
        Map_out, Width_unrolled);

    matrix_permute_kernel<<<permute_grid, PERMUTE_BLOCK_SIZE, 0, streams[s]>>>(
        matmul_output[s], output_chunk, Map_out, current_batch, out_image_size);
}

for (int s = 0; s < stream_count; ++s) {
    cudaStreamSynchronize(streams[s]);
    cudaFree(unrolled[s]);
    cudaFree(matmul_output[s]);
    cudaStreamDestroy(streams[s]);
}
```

Listing 3: req_0 stream implementation excerpt: asynchronous prolog and chunked multi-stream kernel launches.

Table 5: WMMA and PTX MMA parameter sweep results for Conv2.

Conv2 candidate	Batch	Conv1 op time	Conv2 op time	Total op time	Accuracy	Decision
WMMA baseline	100	0.084228 ms	0.156062 ms	0.240290 ms	0.86	Baseline WMMA kernel
WMMA, 4 warps/block, 1 N tile/warp	100	0.085130 ms	0.142167 ms	0.227297 ms	0.86	Kept for full-batch test
WMMA, 4 warps/block, 1 N tile/warp	10000	2.83016 ms	10.1206 ms	12.95076 ms	0.8714	Correct, slower than final
WMMA, 4 warps/block, 2 N tiles/warp	100	0.084157 ms	0.155581 ms	0.239738 ms	0.86	Rejected: extra pressure outweighed A reuse
WMMA K32-style variant	100	0.088777 ms	0.158849 ms	0.247626 ms	0.86	Rejected
Row-tile WMMA variant	100	0.087824 ms	0.160490 ms	0.248314 ms	0.86	Rejected
Shape-specialized 4-warp WMMA	100	0.085260 ms	0.137248 ms	0.222508 ms	0.86	Kept
Shape-specialized 4-warp WMMA	10000	2.76322 ms	9.78287 ms	12.54609 ms	0.8714	Final source
WMMA 8 warps/block plus Conv1 row-window reuse	10000	2.46372 ms	9.81082 ms	12.27454 ms	0.8714	Conv1 improved, Conv2 did not
PTX MMA, corrected A-fragment mapping	100	0.089999 ms	0.141726 ms	0.231725 ms	0.86	Correct but slower than final WMMA
PTX MMA, compact shared stride	100	0.084738 ms	0.138479 ms	0.223217 ms	0.86	Close, but slower than final WMMA
PTX MMA, compact shared stride	10000	2.86904 ms	9.80617 ms	12.67521 ms	0.8714	Correct but slower than final

Table 6: Nsight Compute resource and throughput comparison for Conv2 WMMA kernels.

Kernel/profile	Block size	Grid size	Registers/thread	Static shared memory	SM throughput	Tensor pipe active	DRAM throughput	L2 throughput	Active warps	Eligible warps
Earlier `req_1` WMMA Conv2	32	722500	38	2.048 KB	61.51%	1.43%	not collected	3.78%	not collected	1.026
Final 4-warp WMMA Conv2	128	1807	56	7.616 KB	83.51%	4.89%	11.05%	7.74%	65.50%	1.097

Table 7: Final Conv2 WMMA stall metrics from Nsight Compute.

Final WMMA stall metric	Value
Long scoreboard stalls / issue-active warp	2.955
Short scoreboard stalls / issue-active warp	2.179
Barrier stalls / issue-active warp	1.853
MIO throttle stalls / issue-active warp	2.413

Analysis: the stream implementation is correct and demonstrates stream usage, but the unfused path allocates and processes large unrolled matrices. That memory traffic dominates this small CNN, so streams are kept only as the required individual optimization. They are disabled in the final source because the instructions require a single stream for final performance and competition evaluation.

Synergy: this optimization composes naturally with unfused unrolling pipelines, but not with the final single-stream performance submission.

References: CUDA C++ Programming Guide, Streams and Asynchronous Concurrent Execution; NVIDIA blog, "How to Overlap Data Transfers in CUDA C/C++."

9.2 req_1: Tensor Cores

Folder: `project/m3/req_1`

Artifacts: `m3.out`, `req_1_ncu.ncu-rep`, `req_1_nsys.nsys-rep`, `req_1_ncu_stdout.log`

Google Drive subfolder: INSERT_REQ_1_DRIVE_LINK_HERE.

Theory: Tensor Cores accelerate matrix multiply-accumulate tiles. For convolution, the filter matrix and the implicit im2col matrix form GEMM tiles. WMMA lets each warp compute a $16 \times 16 \times 16$ MMA tile with FP16 inputs and FP32 accumulation.

Rubric note: this implementation uses FP16 WMMA inputs with FP32 accumulation. The README rubric states that full Tensor Core credit requires TF32 and that FP16 Tensor Core usage may receive at most partial Tensor Core credit. I used FP16 WMMA because it preserved the required accuracy for this network while giving the fastest validated competition-safe implementation.

Implementation: `req_1/m3-forward.cu` contains the WMMA implicit-GEMM path used by the final source. The prolog converts Conv2 weights to `half`, the kernel stages A and B tiles in shared memory, calls `wmma::load_matrix_sync` and `wmma::mma_sync`, then scatters the resulting tile back to NCHW output.

```
half* host_mask_half = new half[num_mask_elems];
for (int i = 0; i < num_mask_elems; ++i) {
    host_mask_half[i] = __float2half(host_mask[i]);
}
cudaMalloc(reinterpret_cast<void **>(device_mask_ptr),
           num_mask_elems * sizeof(half));
cudaMemcpy(*device_mask_ptr, host_mask_half,
           num_mask_elems * sizeof(half), cudaMemcpyHostToDevice);

wmma::fragment<wmma::matrix_a, 16, 16, 16,
               half, wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b, 16, 16, 16,
               half, wmma::row_major> b_frag;
wmma::fragment<wmma::accumulator, 16, 16, 16, float> acc_frag;

wmma::fill_fragment(acc_frag, 0.0f);
wmma::load_matrix_sync(a_frag, &smem_a[0][0], 16);
wmma::load_matrix_sync(b_frag, &smem_b[warp_id][0][0][0], 16);
wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);
```

Listing 4: req_1 Tensor Core excerpt: host-side FP16 mask conversion and WMMA multiply-accumulate.

Table 8: req_0 stream optimization performance.

Batch	Conv1 op time	Conv2 op time	Accuracy
100	10.5673 ms	7.60313 ms	0.86
10000	294.158 ms	241.590 ms	0.8714

Table 9: req_1 Tensor Core optimization performance.

Batch	Conv1 op time	Conv2 op time	Accuracy
100	0.085260 ms	0.137248 ms	0.86
10000	2.76322 ms	9.78287 ms	0.8714

Analysis: the final Tensor Core path is much faster than the M2 fused baseline and satisfies the competition matmul requirement. The remaining cost is not pure Tensor Core math; Nsight Compute points to memory/dependency stalls from implicit B-tile gathering, shared-memory staging, synchronization, and output scatter.

Synergy: this optimization is the core of the final submission and synergizes with shape specialization, fixed dimensions, FP16 filter storage, and Conv1 constant-memory/coarsening.

References: CUDA C++ Programming Guide, Warp Matrix Functions; NVIDIA blog, "Programming Tensor Cores in CUDA 9."

10. Optional Optimizations

The project implements at least 10 optional points: `op_0` (1), `op_1` (1), `op_2` (1), `op_3` (3), `op_5` (2 or 4 depending on grading interpretation), and `op_6` (4). These individual folders are intentionally non-stacked implementations relative to the required baselines, so some of them are slower than the final stacked source. The final source selects the subset of ideas that are both accurate and competition-safe.

10.1 op_0: Constant Memory

Folder: `project/m3/op_0`

Artifacts: INSERT_OP_0_ARTIFACT_LIST_OR_DRIVE_LINK_HERE.

Theory: convolution weights are reused across many output pixels. Constant memory can broadcast reads efficiently when threads in a warp read the same address.

Implementation: the mask is stored in module-scope `__constant__ float c_mask[]` and the fused implicit-unroll kernel reads weights from constant memory instead of global memory.

```
static constexpr int kMaxMaskSize = 16384;
__constant__ float c_mask[kMaxMaskSize];

// Prolog for Conv1
cudaMemcpyToSymbol(c_mask, host_mask, mask_bytes);

// Hot loop in Conv1
#pragma unroll
for (int m = 0; m < kConv1MapOut; ++m) {
    const float w = c_mask[m * kConv1Ckk + k];
    #pragma unroll
    for (int cw = 0; cw < kCoarsen; ++cw) {
        acc[m][cw] += w * x_val[cw];
    }
}
```

Listing 5: op_0 constant-memory excerpt: module-scope mask storage and device-side broadcast load.

Table 10: Optional optimization summary results for individual optimization folders.

Folder	Optimization	Batch	Conv1 op time	Conv2 op time	Accuracy
`op_0`	Constant memory	10000	68.2982 ms	63.9242 ms	0.8714
`op_1`	`__restrict__` qualifiers	10000	68.0204 ms	41.1548 ms	0.8714
`op_2`	Loop unrolling	10000	68.0251 ms	41.1668 ms	0.8714
`op_3`	Parameter sweep folder implementation	10000	174.958 ms	107.871 ms	0.8714
`op_5`	FP16 arithmetic	10000	76.4468 ms	46.911 ms	0.8714
`op_6`	Joint register/shared-memory tiling	10000	73.3896 ms	46.2649 ms	0.8714

Analysis: constant memory alone does not overcome the fused baseline's scalar FMA and implicit gather costs, but it is effective in the final Conv1 path because Conv1's mask is tiny and heavily reused. Compared with the M2 fused baseline, this individual optimization did not improve total runtime, but it established a mechanism that later synergized strongly with the specialized Conv1 kernel.

Synergy: combines well with Conv1 fixed-shape indexing and thread coarsening.

References: CUDA C++ Programming Guide, Constant Memory.

10.2 op_1: `__restrict__`

Folder: `project/m3/op_1`

Artifacts: INSERT_OP_1_ARTIFACT_LIST_OR_DRIVE_LINK_HERE.

Theory: `__restrict__` tells the compiler that pointers do not alias, allowing better load scheduling and caching decisions.

Implementation: mask, input, and output pointers are marked as `__restrict__` in hot kernels.

```
__global__ void Conv1ImplicitGemmKernel(const float* __restrict__ input,
                                        float* __restrict__ output,
                                        int batch, int map_out, int channel,
                                        int height, int width, int k_dim);

__global__ void Conv2WmmaImplicitGemmKernel(const half*  __restrict__ mask_half,
                                            const float* __restrict__ input,
                                            float*       __restrict__ output,
                                            int map_out, int channel,
                                            int height, int width, int k_dim,
                                            int height_out, int width_out,
                                            int batch);
```

Listing 6: op_1 restrict excerpt: non-aliasing pointer qualifiers in final hot kernels.

Analysis: Conv2 improves versus the M2 fused baseline, but the scalar FFMA path still lacks Tensor Core acceleration. This optimization synergizes with the final source because the final custom kernels also mark hot pointers as restricted.

Synergy: helps compiler scheduling and complements both scalar and WMMA paths.

References: NVIDIA blog, "CUDA Pro Tip: Optimize for Pointer Aliasing."

10.3 op_2: Loop Unrolling

Folder: `project/m3/op_2`

Artifacts: INSERT_OP_2_ARTIFACT_LIST_OR_DRIVE_LINK_HERE.

Theory: unrolling reduces loop-control overhead and exposes more instruction-level parallelism.

Implementation: hot fixed-size loops are unrolled with pragmas or manual fixed-size loops.

```
float acc[kConv1MapOut][kCoarsen];

#pragma unroll
for (int m = 0; m < kConv1MapOut; ++m) {
    #pragma unroll
    for (int cw = 0; cw < kCoarsen; ++cw) {
        acc[m][cw] = 0.0f;
    }
}

#pragma unroll
for (int k = 0; k < kConv1Ckk; ++k) {
    // load input window values and accumulate
    #pragma unroll
    for (int m = 0; m < kConv1MapOut; ++m) {
        #pragma unroll
```

```
        for (int cw = 0; cw < kCoarsen; ++cw) {
            acc[m][cw] += c_mask[m * kConv1Ckk + k] * x_val[cw];
        }
    }
}
```

Listing 7: op_2 loop-unrolling excerpt: fixed-size Conv1 loops unrolled across output maps and coarsened columns.

Analysis: the individual timing is close to `op_1`, meaning the compiler already handled many simple loops. Loop unrolling remains useful in the final fixed-shape Conv1 and Conv2 kernels, where loop trip counts are known at compile time.

Synergy: works with fixed-shape specialization and register blocking.

References: course optimization lectures and textbook discussion of loop unrolling.

10.4 op_3: Parameter Sweep

Folder: `project/m3/op_3`

Artifacts: INSERT_OP_3_ARTIFACT_LIST_OR_DRIVE_LINK_HERE.

Theory: block sizes, tile shapes, and coarsening determine occupancy, shared-memory pressure, coalescing, and reuse. The best setting depends on the layer shape.

Implementation: I swept Conv2 WMMA parameters including warps per block, N tiles per warp, K tile shape, row/column mapping, fixed-shape indexing, and PTX MMA versus WMMA.

```
static constexpr int kWmmaM = 16;
static constexpr int kWmmaN = 16;
static constexpr int kWmmaK = 16;

// Final selected configuration after sweep
static constexpr int kWarpsPerBlock = 4;
static constexpr int kNTilesPerWarp = 1;

static constexpr int kConv2BlockSize =
    kWarpsPerBlock * 32;
static constexpr int kConv2TilesPerBlock =
    kWarpsPerBlock * kNTilesPerWarp;
```

Listing 8: op_3 parameter-sweep knobs: compile-time constants used to compare WMMA block and tile configurations.

Analysis: multiple WMMA block/tile settings were tested, and the submitted kernel uses a 4-warp, 1-N-tile configuration together with shape specialization. A 2-N-tile-per-warp variant increased pressure and was slower. K32 and row-tile variants were slower. PTX MMA was correct after fixing fragment mapping, but full-batch timing remained slightly slower than the final WMMA path.

The `op_3` folder result in the optional summary table is the required individual folder implementation. The broader sweep tables above include the final-candidate experiments used to select the submitted parameters.

Synergy: this is the tuning layer that selected parameters used by the final `req_1`-style WMMA path.

References: CUDA occupancy and memory hierarchy lectures; Nsight Compute profiling lecture.

10.5 op_5: FP16 Arithmetic

Folder: `project/m3/op_5`

Artifacts: INSERT_OP_5_ARTIFACT_LIST_OR_DRIVE_LINK_HERE.

Theory: FP16 reduces bandwidth and feeds Tensor Cores. Accuracy depends on whether accumulation remains FP32.

Implementation: the individual FP16 variant stores weights as `__half`. The final Conv2 stores WMMA inputs as FP16 and accumulates in FP32.

```
const int num_mask_elems = Map_out * Channel * K * K;
half* host_mask_half = new half[num_mask_elems];

for (int i = 0; i < num_mask_elems; ++i) {
    host_mask_half[i] = __float2half(host_mask[i]);
}

cudaMalloc(reinterpret_cast<void **>(device_mask_ptr),
           num_mask_elems * sizeof(half));
cudaMemcpy(*device_mask_ptr, host_mask_half,
           num_mask_elems * sizeof(half), cudaMemcpyHostToDevice);
delete[] host_mask_half;

wmma::fragment<wmma::accumulator, 16, 16, 16, float> acc_frag;
wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);
```

Listing 9: op_5 FP16 excerpt: host conversion to half and FP32 accumulation through WMMA.

Analysis: the final Tensor Core path is the useful synergy: FP16 is used where WMMA requires it, while accumulation and output remain FP32. Accuracy matches the expected `0.8714` on batch 10000.

Synergy: enables the WMMA Tensor Core path while preserving accuracy via FP32 accumulation.

References: NVIDIA mixed-precision programming documentation; CUDA Math API for `half` and `half2`.

10.6 op_6: Joint Register and Shared-Memory Tiling

Folder: `project/m3/op_6`

Artifacts: INSERT_OP_6_ARTIFACT_LIST_OR_DRIVE_LINK_HERE.

Theory: each thread holds a register sub-tile while shared memory stages A and B tiles. This increases arithmetic intensity because each shared load feeds multiple register accumulators.

Implementation: `op_6/m3-forward.cu` computes a `TILE_M x TILE_N` output tile with per-thread `COARSEN_M x COARSEN_N` accumulators.

```
#define TILE_M     32
#define TILE_N     64
#define TILE_K     16
#define COARSEN_M  4
#define COARSEN_N  4

__shared__ float tileA[TILE_M][TILE_K];
__shared__ float tileB[TILE_K][TILE_N];

float acc[COARSEN_M][COARSEN_N] = {0.0f};

for (int kt = 0; kt < num_k_tiles; ++kt) {
    tileA[r][c] = mask[(size_t)global_m * numAColumns + global_k];
    tileB[r][c] = input[(size_t)b * Channel * Height * Width
                        + cc * Height * Width
                        + (h_out + p) * Width + (w_out + q)];
    __syncthreads();

    #pragma unroll
    for (int k = 0; k < TILE_K; ++k) {
        float a_reg[COARSEN_M];
        float b_reg[COARSEN_N];

        #pragma unroll
        for (int i = 0; i < COARSEN_M; ++i)
            a_reg[i] = tileA[threadIdx.y * COARSEN_M + i][k];

        #pragma unroll
        for (int j = 0; j < COARSEN_N; ++j)
            b_reg[j] = tileB[k][threadIdx.x * COARSEN_N + j];
```

```
31
32          #pragma unroll
33          for (int i = 0; i < COARSEN_M; ++i)
34              #pragma unroll
35              for (int j = 0; j < COARSEN_N; ++j)
36                  acc[i][j] += a_reg[i] * b_reg[j];
37      }
38      __syncthreads();
39  }
```

Listing 10: op_6 joint register/shared-memory tiling excerpt: cooperative shared loads and per-thread register subtile.

Analysis: the individual JRT implementation reduces some memory stalls compared with simple fused kernels, but it uses more registers. It informed the final preference for keeping accumulators in registers, while the active final Conv2 uses Tensor Cores instead of scalar register tiling.

Synergy: conceptual foundation for accumulator-heavy tiling in the final Conv1 and Conv2 kernels.

References: ECE 508 joint register/shared-memory tiling lecture; Carl Pearson profiling lecture notes.

11. Synergy and Final Selection

The final submission combines the ideas that are both fast and competition-safe:

- Implicit unrolling avoids explicit im2col allocation.
- Tiled matmul satisfies the competition requirement.
- FP16 Tensor Core inputs reduce bandwidth while FP32 accumulation preserves accuracy.
- The final 4-warp, 1-N-tile-per-warp WMMA configuration balanced scheduling and resource pressure.
- Shape specialization reduced fixed-dimension indexing overhead in Conv2.
- Constant memory and coarsening are kept for Conv1, where they are a natural fit.

Streams are intentionally not used in the final file because the instructions require single-stream final performance evaluation. The final source keeps Conv2 on the tiled implicit-GEMM WMMA path throughout.

12. Verification Commands

Build:

```
./run.sh build
```

Normal full-batch run:

```
sbatch --wait --job-name=M3_WMMASPEC_10K --output=m3_wmmaspec_10k.slurm.out --error=m3_wmmaspec_10k.slurm.err --partition=gpuA40x4 --mem=12G --
      nodes=1 --ntasks-per-node=1 --cpus-per-task=1 --constraint=projects --gpus-per-node=1 --gpu-bind=closest --account=bche-delta-gpu -t
      00:05:00 --wrap='srun ./m3 10000 > m3_wmmaspec_10k.out'
```

Competition-mode run:

```
sbatch --wait --job-name=M3_COMP --output=m3_competition.slurm.out --error=m3_competition.slurm.err --partition=gpuA40x4 --mem=12G --nodes=1 --
      ntasks-per-node=1 --cpus-per-task=1 --constraint=projects --gpus-per-node=1 --gpu-bind=closest --account=bche-delta-gpu -t 00:05:00 --wrap=
      'srun ./m3 --competition > m3_competition.out'
```

Final WMMA Nsight Compute command:

```
sbatch --wait --job-name=M3_WMMASPEC_NCU --output=m3_wmmaspec_ncu.slurm.out --error=m3_wmmaspec_ncu.slurm.err --partition=gpuA40x4 --mem=12G --
      nodes=1 --ntasks-per-node=1 --cpus-per-task=1 --constraint=projects,perf,nvperf --gpus-per-node=1 --gpu-bind=closest --account=bche-delta-
      gpu -t 00:10:00 --wrap='srun ncu --kernel-name "regex:Conv2WmmaImplicitGemmKernel" --launch-count 1 --metrics sm__throughput.avg.
      pct_of_peak_sustained_elapsed,dram__throughput.avg.pct_of_peak_sustained_elapsed,lts__throughput.avg.pct_of_peak_sustained_elapsed,
      sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_active,smsp__warps_active.avg.pct_of_peak_sustained_active,smsp__warps_eligible.avg.
      per_cycle_active,smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio,
      smsp__average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio,smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio,
      smsp__average_warps_issue_stalled_mio_throttle_per_issue_active.ratio,launch__registers_per_thread,launch__shared_mem_per_block_static --
      csv -f -o m3_wmmaspec_conv2_ncu ./m3 100 > m3_wmmaspec_ncu.out'
```

13. Conclusion

The final code meets the M3 performance requirement and competition warmup guardrails with a rule-compliant implicit-GEMM, tiled Tensor Core Conv2 implementation. The best final run is `12.54609 ms` total for batch 10000, and `--competition` mode reports `12.58574 ms` total with the expected `0.8714` accuracy. The main remaining opportunity for a faster implementation is deeper PTX MMA tuning of shared-memory layout, tile scheduling, and accumulator handling while preserving correctness.